

Generación y chequeo

Objetivos

- Conocimiento de técnicas de programación en Prolog.
- Generación y chequeo como método de especificación ejecutable.
- Desafíos en cuanto a la eficiencia.
- Desarrollo incremental de 3 ejemplos:
 - Sorting
 - N-Reinas
 - Coloración de mapas

Generación y chequeo

- Técnica para diseñar y programar algoritmos.
- Un proceso genera soluciones candidatas al problema.
- Otro proceso chequea los candidatos, intentando encontrar aquel o aquellos que resuelvan el problema.

Generación y chequeo

Forma general:

```
solucion(X) :-  
    generacion(X),  
    chequeo(X).
```

Método:

- generacion(X) retorna candidatos X tales que X tiene el formato de una eventual solución.
- chequeo(X) verifica si X satisface todas las condiciones de una solución.

Sorting

Deseamos ordenar una lista de n números.

Describimos las propiedades de una solución.

Sorting

Deseamos ordenar una lista de n números.

Describimos las propiedades de una solución.

```
ordenar(L,P) :-  
    permutacion(L,P),  
    ordenada(P).
```

Sorting

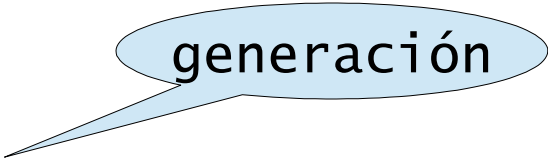
Deseamos ordenar una lista de n números.

Describimos las propiedades de una solución.

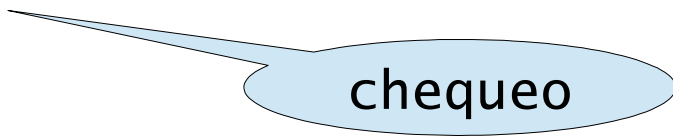
```
ordenar(L,P) :-
```

```
    permutacion(L,P),
```

```
    ordenada(P).
```



generación



chequeo

Sorting: permutación

```
%permutacion(+L,?P) <- La lista P es una  
                        permutación de la  
                        lista L.
```

Sorting: permutación

`%permutacion(+L, ?P) <-` La lista P es una permutación de la lista L.

Idea:

- Seleccionamos un 1er elemento de la solución (cualquier elemento de la lista).
- El resto es una permutación de los restantes elementos de la lista.

Sorting: permutación, select

```
%select(?E, +L, ?LSinE) <- E es un elemento de  
                           L y LSinE es L sin  
                           el elemento E.
```

Sorting: permutación, select

`%select(?E,+L,?LSinE) <-` E es un elemento de L y LSinE es L sin el elemento E.

```
?- select(a,[b,a,c],LSinE).  
LSinE = [b,c] ;  
false.
```

```
?- select(a,[b,a,c,a],LSinE).  
LSinE = [b,c,a] ;  
LSinE = [b,a,c] ;  
false.
```

```
?- select(E,[a,b],LSinE).  
E = a,  
LSinE = [b] ;  
E = b,  
LSinE = [a] ;  
false.
```

Sorting: permutación, select

`%select(?E,+L,?LSinE) <-` E es un elemento de L
y LSinE es L sin el
elemento E.

`select(X,[X|Xs],Xs).`

`select(X,[Y|Xs],[Y|R]) :-
 select(X,Xs,R).`

Sorting: ordenada

%ordenada(+L) <- La lista de números L está
ordenada en forma ascendente,
 $l_i \leq l_{i+1}$, $i=1, \dots, n-1$.

Sorting: ordenada

`%ordenada(+L) <-` La lista de números `L` está
ordenada en forma ascendente,
 $l_i \leq l_{i+1}$, $i=1, \dots, n-1$.

`ordenada([_]).`

`ordenada([X,Y|R]) :-`
 $X \leq Y$,
 `ordenada([Y|R]).`

Sorting: ineficiencia

```
?- ordenar([9,8,7,6,5,4,3,2,1],L).
```

```
L = [1,2,3,4,5,6,7,8,9] .
```

```
?- profile(ordenar([9,8,7,6,5,4,3,2,1],L)).
```

```
L = [1,2,3,4,5,6,7,8,9] .
```

```
?- profile(sort([9,8,7,6,5,4,3,2,1],L)).
```

```
L = [1,2,3,4,5,6,7,8,9] .
```

245 seg., 2.517.400 =<, 1.561.417 ordenada

```
?- profile(ordenar([20,19,18,17,16,15,...,9,8,7,6,5,4,3,2,1],L)).
```

```
Action (h for help) ? abort
```

```
% Execution Aborted
```

0.61 seg., 623.529 =<, 362.880 ordenada

```
?- profile(sort([20,19,18,17,16,15,...,9,8,7,6,5,4,3,2,1],L)).
```

```
L = [1,2,3,4,5,6,7,8,9|...].
```

Ineficiencia

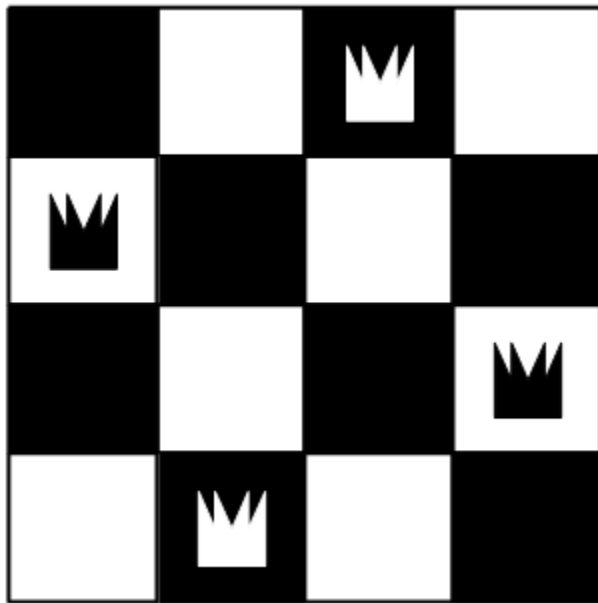
Problema: eficiencia.

Posible solución:

Una técnica para optimizar programas consiste en “empujar” el chequeo dentro del generador lo más profundo posible.

Problema de las N-Reinas

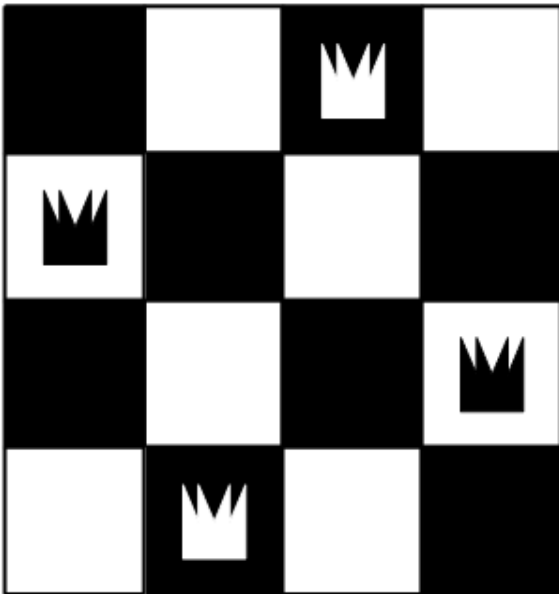
Distribuir N reinas en un tablero de $N \times N$, de forma que todo casillero del tablero quede atacado por una reina, y ninguna reina sea atacada por otra.



Una solución al
problema de
las 4-Reinas

Problema de las N-Reinas

Representación:



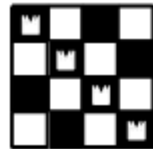
2	4	1	3
---	---	---	---

Lista de N elementos:

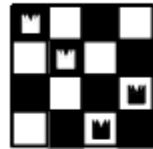
El valor del i -ésimo elemento en la lista es la fila donde se coloca la reina de la columna i

La representación de datos ya incorpora parte de las restricciones!

Problema de las N-Reinas: generación

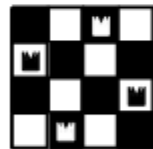


1	2	3	4
---	---	---	---



1	2	4	3
---	---	---	---

⋮



2	4	1	3
---	---	---	---

⋮



3	1	4	2
---	---	---	---

Problema de las N-Reinas

```
reinas(N, Qs) :-  
    primeros(N, Ns),  
    permutacion(Ns, Qs),  
    segura(Qs).
```

Problema de las N-Reinas

```
reinas(N, Qs) :-  
    primeros(N, Ns),  
    permutacion(Ns, Qs),  
    segura(Qs).
```

%primeros(+N, ?Ns) <- inicializa la lista Ns
con N posiciones diferentes, entonces
tenemos una reina en cada columna y además
todas están en diferentes filas.

Problema de las N-Reinas

```
reinas(N,Qs) :-  
    primeros(N,Ns),  
    permutacion(Ns,Qs),  
    segura(Qs).
```

```
segura([Q|Qs]) :-  
    no_ataca(Q,Qs),  
    segura(Qs).
```

```
segura([]).
```

```
no_ataca(X,Xs) :- no_ataca(X,1,Xs).
```

```
no_ataca(_,_,[]).
```

```
no_ataca(X,N,[Y|Ys]) :-  
    X \= Y+N,  
    X \= Y-N,  
    N1 is N+1,  
    no_ataca(X,N1,Ys).
```

Problema de las N-Reinas

Problema:

Chequea todas las permutaciones (muy ineficiente).

Mejora:

Chequear, a medida que se arma la estructura, si la ubicación de la reina es legal.

Problema de las N-Reinas

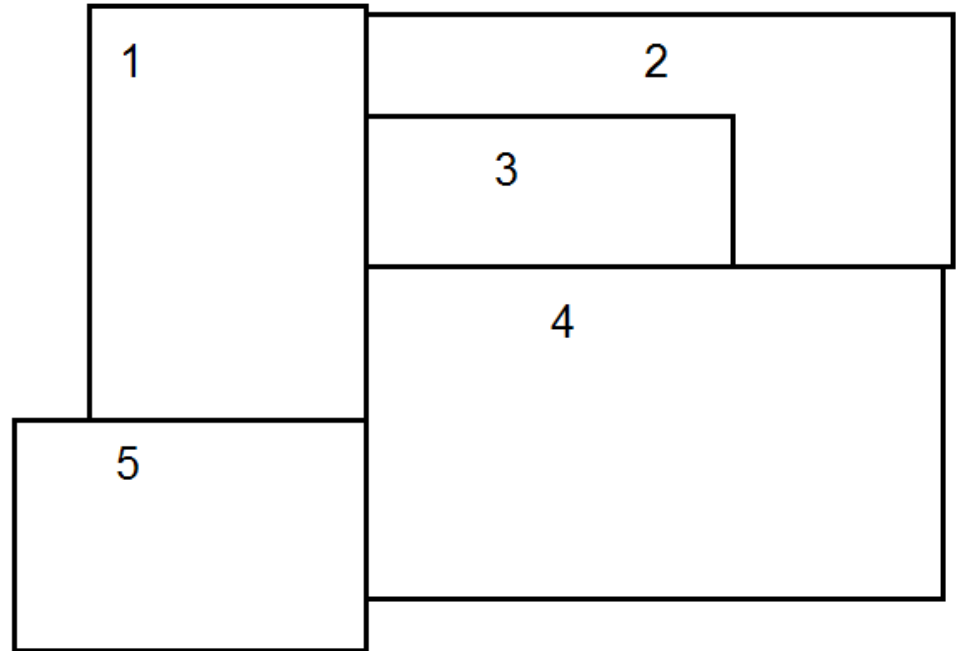
```
reinas(N,Qs) :-  
    primeros(N,Ns),  
    reinas(Ns,[],Qs). % agrego un argumento como acumulador  
  
reinas(SinColocar,Seguras,Qs) :-  
    select(Q,SinColocar,SinColocar1),  
    no_ataca(Q,Seguras),  
    reinas(SinColocar1,[Q|Seguras], Qs).  
  
reinas([],Qs,Qs). % paso base: el acumulador contiene la solución  
  
select :- ...  
  
no_ataca :- ...
```

Coloración de mapas

La figura representa un mapa.
Se debe colorear de modo que
regiones adyacentes tengan
distinto color.

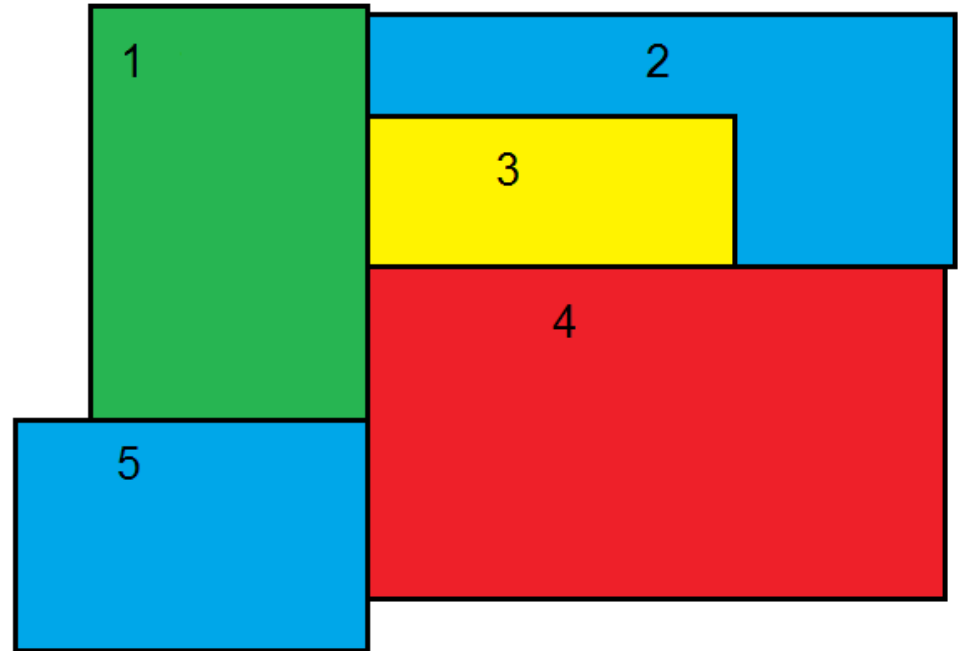
Conjetura: todo mapa (con
regiones continuas) se puede
colorear con 4 colores.

Fue probada por Appel y
Haken (1976).



Coloración de mapas

Coloración con 4 colores.



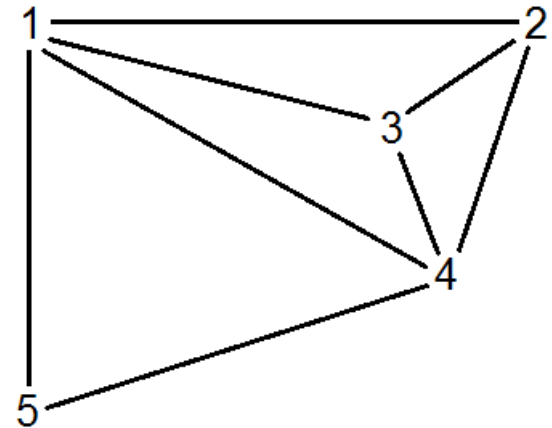
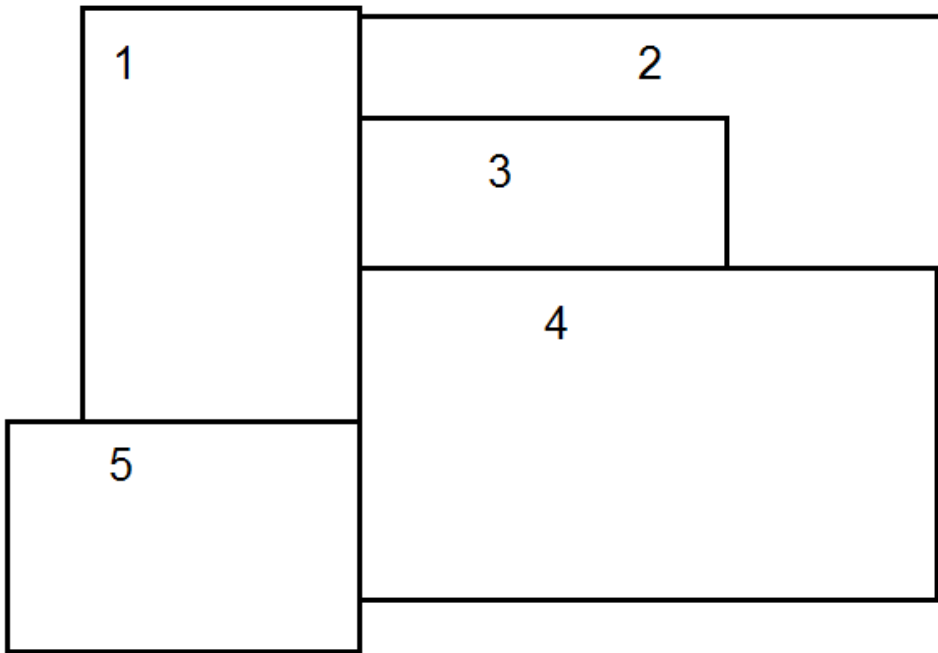
Coloración de mapas

Seguiremos un esquema de generación y chequeo.

- Generación: pintamos todas las regiones con algún color
- Chequeo: no hay regiones vecinas con el mismo color

Pero primero debemos decidir cómo vamos a representar los datos.

Coloración de mapas: representación



Las relaciones de adyacencia de las regiones determinan un grafo. Cada región es un vértice, hay arista entre todo par de regiones adyacentes.

Coloración de mapas: representación

Regiones

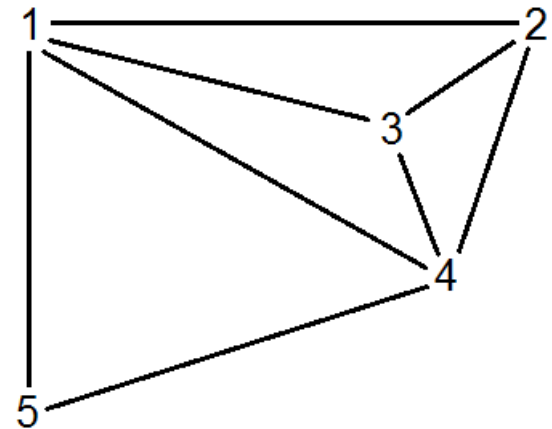
Declaramos las regiones:

```
regiones([1, 2, 3, 4, 5]).
```

Vecinos

Relación para las aristas:

```
vecino(1, 2).    vecino(2, 4).  
vecino(1, 3).    ...  
vecino(1, 4).  
vecino(1, 5).
```



Coloración de mapas: representación

Regiones

Declaramos las regiones:

```
regiones([1, 2, 3, 4, 5]).
```

Vecinos (variante)

Listas de adyacencias:

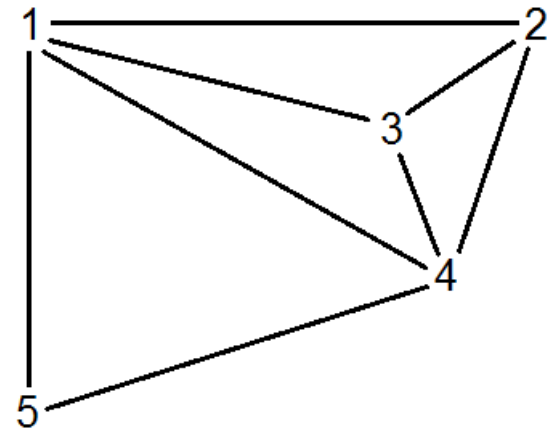
```
vecinos(1, [2, 3, 4, 5]).
```

```
vecinos(2, [1, 3, 4]).
```

```
vecinos(3, [1, 2, 4]).
```

```
vecinos(4, [1, 2, 3, 5]).
```

```
vecinos(5, [1, 4]).
```



Coloración de mapas: representación

Colores

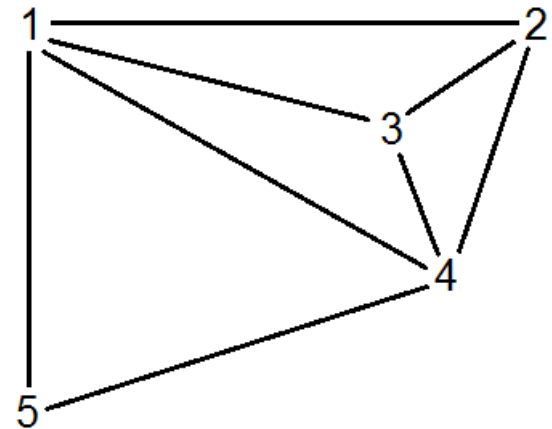
Definimos una paleta de colores:

```
colores([rojo, verde, amarillo,  
azul, violeta, gris]).
```

Mapa coloreado

Una lista de pares (Región,Color):

```
[(1, rojo), (2, verde), ...]
```



Coloración de mapas: generación y chequeo

```
mapa_coloreado(MapaColor) :-  
    genera(MapaColor),  
    sin_conflicto(MapaColor).
```

Coloración de mapas: generación y chequeo

```
genera(MapaColor) :-  
    regiones(R),  
    colores(C),  
    pintar(R,C,MapaColor).
```

```
pintar([],_,[]).
```

```
pintar([R|Rs],C,[(R,C1)|RM]) :-  
    member(C1,C),  
    pintar(Rs,C,RM).
```

Coloración de mapas: generación y chequeo

```
sin_conflicto([_]).
```

```
todos_color_distinto(_, [], _).
```

```
sin_conflicto([(R,C)|RM]) :-  
    sin_conflicto(R,C,RM),  
    sin_conflicto(RM).
```

```
todos_color_distinto(C, [V|Vs], RM) :-  
    not_member((V,C), RM),  
    todos_color_distinto(C, Vs, RM).
```

```
sin_conflicto(R,C,RM) :-  
    vecinos(R,Vs),  
    todos_color_distinto(C,Vs,RM).
```

```
not_member(_, []).
```

```
not_member(X, [Y|Resto]) :-  
    X \= Y,  
    not_member(X, Resto).
```

Coloración de mapas: generación y chequeo

- Se construyó una MALA solución !!
- Se puede mejorar mucho, incluyendo chequeo en la generación.
- El objetivo es fallar lo antes posible si una asignación de color no sirve.
- Ejercicio: Mejorar la solución propuesta (chequear por adyacentes al momento de pintar).

(ver solución en sección *Generate-and-Test* de *The Art of Prolog*)